

13a — Algorithmen modellieren

Vom Problem zum Plan: Pseudocode und Flussdiagramme

HTW Berlin – Wirtschaftsingenieurwesen

SoSe 2026

Agenda

1. Was ist ein Algorithmus? Die 5 Eigenschaften
2. Pseudocode – die Sprache vor der Sprache
3. Flussdiagramme – Kontrollfluss sichtbar gemacht
4. Ablaufverfolgung: den Plan von Hand durchspielen
5. **Klassiker: Alltagsprozess als Flussdiagramm** – live an der Tafel

Lernziel: Abläufe **auf Papier** präzise machen (Pseudocode, Diagramm, Trace) – lange bevor eine einzige Zeile Python getippt wird.

Heute ohne Python: Ein Algorithmus ist **sprachunabhängig**. Wir arbeiten mit Stift, Papier und Tafel.

Wozu Algorithmen modellieren?

Ohne Plan: direkt drauflos tippen

- Die Regel steht nur **im Kopf** – niemand sonst kann sie prüfen
- Sonderfälle fallen erst auf, wenn das Programm sie falsch behandelt
- Ist der Fehler in der **Idee** oder im **Tippen**? Nicht zu unterscheiden

Mit Plan: erst die Idee festhalten

```
ALGORITHMUS Versand(summe)
```

```
    WENN summe >= 100 DANN
```

```
        RÜCKGABE 0.00
```

```
    SONST
```

```
        RÜCKGABE 4.95
```

Erst denken, dann tippen – weniger Bugs, sprachneutral, im Team diskutierbar. Und: dieser Plan ist noch **kein Python** – er gilt für jede Sprache.

Was ist ein Algorithmus?

- Eine **endliche Folge** präzise definierter Schritte
- Löst ein **klar umrissenes Problem** – für **beliebige** gültige Eingaben, nicht nur für ein Beispiel
- **Sprachunabhängig**: erst die Idee, dann Python/Java/...

Algorithmus vs. Programm

Algorithmus	Programm
Lösungsidee	konkrete Umsetzung
Pseudocode, Diagramm	Python, Java, ...
auf Papier nachvollziehbar	auf dem Rechner ausführbar

Alltags-Algorithmen: Kochrezept, Wegbeschreibung, Bedienungsanleitung, die Versandregel bei Veggie Soles.

Die 5 Eigenschaften eines Algorithmus

Eigenschaft	bedeutet
Endlichkeit	endet nach endlich vielen Schritten
Eindeutigkeit	jeder Schritt präzise, keine Interpretation
Ausführbarkeit	jeder Schritt ist mechanisch machbar
Ein-/Ausgabe	definierte Eingaben, mindestens eine Ausgabe
Determinismus	gleiche Eingabe → gleicher Ablauf, gleiches Ergebnis

Das ist die **Qualitätskontrolle vor dem Code**: Verletzt mein Plan eine Eigenschaft, lohnt sich das Tippen noch nicht.

Unter der Haube: die Eigenschaften ernst genommen

Jede Eigenschaft lässt sich **verletzen** – daran erkennt man, was sie leistet:

Anweisung	verletzt
“nach Geschmack würzen” (Rezept)	Eindeutigkeit
SOLANGE $b \neq 0$: ... – aber b wird nie geändert	Endlichkeit
“wähle die größte Primzahl” (existiert nicht)	Ausführbarkeit
Ergebnis berechnen, aber nie ausgeben	Ein-/Ausgabe
“wirf eine Münze, bei Kopf ...”	Determinismus

Endlichkeit ist kein Glaube, sondern ein Argument: Eine Größe muss pro Wiederholung **strikt** Richtung Abbruch wandern – sonst dreht sich der Plan für immer im Kreis.

Pseudocode - die Sprache vor der Sprache

```
ALGORITHMUS Versand(summe)
```

```
    WENN summe >= 100 DANN
```

```
        RÜCKGABE 0.00
```

```
    SONST
```

```
        RÜCKGABE 4.95
```

- Halb Deutsch, halb Code: **Logik festhalten**, Syntaxdetails weglassen
- **Kein Standard** – wir nutzen die Konventionen aus Notebook 13a
- Struktur durch **Einrückung**, Zuweisung mit $\leftarrow$, Schlüsselwörter GROSS

Pseudocode ist für **Menschen** geschrieben – eine Programmiersprache für Menschen *und* Maschinen.

Unter der Haube: die Pseudocode-Bausteine

Baustein	Notation	Bedeutung
Kopf	ALGORITHMUS Name(param)	Name + Eingaben
Zuweisung	summe $\leftarrow$ 0	Wert in eine Variable legen
Ausgabe	AUSGABE summe / RÜCKGABE summe	Ergebnis verlassen den Algorithmus
Verzweigung	WENN ... DANN ... SONST	zwei Wege, eine Frage
Wiederholung	SOLANGE bedingung:	Block erneut, solange die Frage "ja" ist
Zählschleife	FÜR i VON 1 BIS n:	Block einmal pro Zählwert
Rest	a MOD b	Rest der ganzzahligen Division

Mehr Bausteine gibt es nicht. **Sequenz, Verzweigung, Wiederholung** – jeder Algorithmus der Welt besteht aus diesen dreien.

Die drei Grundstrukturen

Struktur	Frage, die sie beantwortet
Sequenz	Was passiert nacheinander?
Verzweigung	Welcher von zwei Wegen?
Wiederholung	Was passiert mehrfach?

ALGORITHMUS Bestellbon(preis, menge)

gesamt $\leftarrow$ preis * menge (Sequenz)

WENN gesamt $\geq$ 100 DANN versand $\leftarrow$ 0.00 (Verzweigung)

SONST versand $\leftarrow$ 4.95

AUSGABE gesamt + versand (Sequenz)

Jede Sprache hat eigene Schlüsselwörter – die **Struktur** bleibt identisch.

Ablaufverfolgung: den Plan von Hand durchspielen

Ein Plan ist erst verstanden, wenn Sie ihn **als Rechner** abarbeiten.

ALGORITHMUS Tausche(a, b)

hilf $\leftarrow$ a

a $\leftarrow$ b

b $\leftarrow$ hilf

Schritt	a	b	hilf
Start	3	7	-
hilf $\leftarrow$ a	3	7	3
a $\leftarrow$ b	7	7	3
b $\leftarrow$ hilf	7	3	3

Trace-Tabelle: Spalte pro Variable, Zeile pro Schritt.

► **Live-Demo** - 01_zuweisung_trace.py: dieselbe Zuweisungsfolge, vom Rechner Schritt für Schritt mitgeschrieben. Die Tafel-Tabelle und die Ausgabe müssen **identisch** sein.

✎ **Sofort ausprobieren** - Notebook 13a: Übung 1.2 (*welche Eigenschaft ist verletzt?*) und die Trace-Tabelle zum Bestellbon-Pseudocode von eben (*preis = 89.95, menge = 1*).

Flussdiagramme - die Bausteine

Symbol	Bedeutung
Oval	Start / Ende
Rechteck	Aktion (Zuweisung, Ausgabe)
Raute	Verzweigung: Frage mit ja/nein-Ausgang
Pfeil	wohin geht es als Nächstes?

- Visuelle Sprache, **sprachneutral** – lesbar auch für Nicht-Programmierende
- Stärke gegenüber Pseudocode: **Verzweigungen und Rückwege** sieht man sofort

Im Notebook 13a zeichnen wir Flussdiagramme mit *Mermaid* direkt in Jupyter.

Unter der Haube: der Pfeil macht die Struktur

Was Sie im Diagramm sehen

Grundstruktur

Rechteck-Kette, Pfeile immer abwärts

Sequenz

Raute, zwei Pfeile, die sich später wieder treffen

Verzweigung

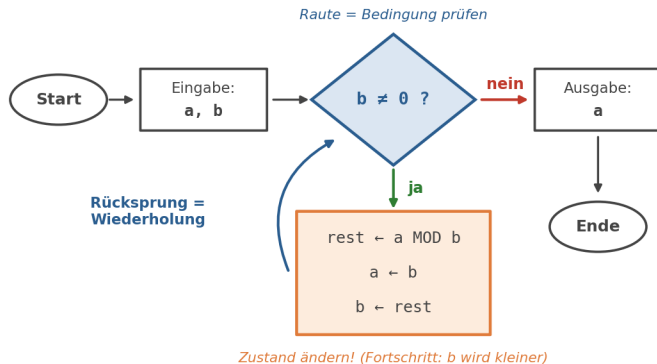
Raute mit einem Pfeil, der **zurück nach oben** zeigt

Wiederholung

- Der **Rücksprung-Pfeil** ist der Trick: dieselbe Raute wird erneut geprüft – nur mit **veränderten** Werten
- Verändert der Rundweg nichts, endet der Plan nie

Ohne Rücksprung-Pfeil kann ein Diagramm **niemals** endlos laufen. Mit Rücksprung brauchen Sie ein Endlichkeits-Argument.

Beispiel: ggT nach Euklid - als Diagramm gelesen



Der älteste Algorithmus der Welt (ca. 300 v. Chr.) – und er passt auf eine halbe Folie. Lesen Sie ihn mit der Symboltabelle: Oval, Rechteck, Raute, **Rücksprung**.

Unter der Haube: Euklid von Hand – Trace für 48 und 18

Veggie Soles bündelt 48 und 18 Kartons **gleich groß und sortenrein** – die größte Bündelgröße ist der ggT.

Prüfung	a	b	$b \neq 0?$	$\text{rest} \leftarrow a \bmod b$
1	48	18	ja	12
2	18	12	ja	6
3	12	6	ja	0
4	6	0	nein	– Ausgabe: 6

- b fällt strikt: $18 \rightarrow 12 \rightarrow 6 \rightarrow 0$ – das **Endlichkeits-Argument**

Sie haben Euklid gerade **ausgeführt** – ohne eine Zeile Python. Genau das meint “sprachunabhängig”.

Klassiker: Alltagsprozess als Flussdiagramm - die Aufgabe

Bei Veggie Soles kommt eine **Retoure** an. Modellieren Sie die Bearbeitung – auf Papier, ohne Rechner:

- Ware unbenutzt? → Gutschrift. Getragen? → Rücksendung an Kundin
- Rücksendefrist von 30 Tagen überschritten? → Ablehnung
- Solange Artikel im Paket liegen: jeden einzeln prüfen

Abzugeben (Papier)

1. **Pseudocode** des Ablaufs
2. **Flussdiagramm** desselben Ablaufs
3. **Trace-Tabelle** für ein Paket mit 2 Artikeln
4. Ein Satz je Eigenschaft: Warum ist Ihr Plan endlich, eindeutig, ausführbar, deterministisch, mit Ein-/Ausgabe?

► **Live-Modellierung an der Tafel** - Referenz: 90_klassiker_loesung.md

Teilziele:

1. Ein-/Ausgabe zuerst festlegen: Was geht rein, was kommt raus?
2. Die Entscheidungen finden - jede wird **eine Raute**
3. Die Wiederholung finden ("jeden Artikel") - **Rücksprung-Pfeil**
4. Endlichkeits-Argument benennen: Welche Größe fällt strikt?
5. Gegenprobe: Trace mit 2 Artikeln, einer davon getragen

✎ **Zum Selberlösen** - Handout "*Klassiker 13a: Alltagsprozess als Flussdiagramm*" (Klassiker/13a_Klassiker_Flussdiagramm.pdf). Vertiefung: Notebook 13a, Kap. 3-4.

Cheat Card

Aufgabe	Notation
Kopf / Ergebnis	ALGORITHMUS $f(x)$ · RÜCKGABE wert
Zuweisung	$x \leftarrow 5$
Verzweigung	WENN c DANN ... SONST ...
Wiederholung	SOLANGE c : · FÜR i VON a BIS b :
Diagramm	Oval = Start/Ende · Rechteck = Aktion Raute = Frage · Rücksprung = Wiederholung
Kontrolle	Trace-Tabelle + die 5 Eigenschaften
Klassiker heute	Retoure $\rightarrow$ Pseudocode $\rightarrow$ Diagramm

Faustregel: erst modellieren, dann prüfen (Trace + 5 Eigenschaften) – **dann** tippen.

Ausblick: was aus dem Plan wird

Ihre Pläne enthalten schon Bausteine, für die uns noch die Python-Syntax fehlt:

Baustein von heute	kommt in
WENN ... DANN ... SONST	Verzweigungen (FS 09)
SOLANGE ... / FÜR ...	Schleifen (FS 10)
ALGORITHMUS $f(x)$ / RÜCKGABE	Funktionen (FS 07)

- Das Übersetzen wird **mechanisch** sein: eine Tabelle, jede Pseudocode-Zeile hat genau ein Gegenstück
- **Versprechen für FS 10:** Dann holen wir das Euklid-Flussdiagramm wieder hervor – und programmieren es in fünf Zeilen

Das Denken passiert **heute**. Später kommt nur noch das Tippen dazu.

Heute geübt

- ✓ Algorithmus definiert und vom Programm abgegrenzt
- ✓ Die 5 Eigenschaften – samt Gegenbeispielen als Prüfliste
- ✓ Pseudocode lesen und schreiben, alle Bausteine
- ✓ Die drei Grundstrukturen: Sequenz, Verzweigung, Wiederholung
- ✓ Flussdiagramme: Raute = Frage, Rücksprung = Wiederholung
- ✓ Trace-Tabellen – Algorithmen von Hand ausführen (inkl. Euklid)
- ✓ **Klassiker Retoure** an der Tafel modelliert

Ihre Aufgaben bis zur nächsten Woche:

- Klassiker **Alltagsprozess** eigenständig modellieren (Handout, Papier!)
- “Sofort ausprobieren“-Aufgaben im Notebook 13a
- Eigenes Kochrezept als Flussdiagramm zeichnen – und die 5 Eigenschaften prüfen